

Csound in the MetaVerse (2026): Personalizing Instruments, Worlds, and Live Sound in CsoundMeta

Strong Bear¹ and Richard Boulanger²

¹ Independent Researcher

`strong@bear.ceo`

² Berklee College of Music, Boston, MA, USA

`rboulanger@berklee.edu`

Abstract. CsoundMeta is an immersive multiplayer environment, built in Unity for Meta Quest headsets, that lets performers touch, shape, and share Csound instruments as physical objects in a shared virtual or physical space. Since the system’s Orbs, Wanderers, and AI-generated Worlds were first introduced, our central goal has shifted from handing performers a curated palette to play with, toward giving every Csounder the tools to build that palette themselves. This paper documents three developments that carry that goal forward. First, performers can now bring their own Csound work into CsoundMeta from further away than the headset itself: any `.csd` file becomes an importable sound for an Orb or Wanderer, Cabbage-authored `.snaps` files import as ready-to-use presets, and a companion web application now lets a performer upload an instrument, complete with its samples, from a desktop browser, with a Dr.C-based assistant on hand to help design, modify, and debug a Csound instrument that will not yet compile. Second, the AI-generated Worlds that once came from a fixed, curated set can now be prompted, remixed, and renamed from inside the headset itself, and repositioned and rotated so a stage arrangement rehearsed at home lines up exactly once the same World is loaded on the performance stage, and every performer, whether standing shoulder to shoulder or joining from across the world, lands in the same generated room and can hand instruments to one another there. Third, and perhaps most consequential for ensemble work, we rebuilt the live audio pipeline around a native engine that carries the microphone-to-effect-to-speaker signal path outside Unity’s own audio thread, fast enough that we now rely on it for real-time capture, playback, and processing shared live across every client in a session rather than kept local to one, and judge it, as musicians do, by ear. Around that engine we added a dedicated, proximity-gated Mic instrument that can also serve the whole session as one shared microphone with automatic handover between performers, an overdub-capable sampler that records from any source in the patch and shares every take across the room instead of keeping it local, and a separate spatialized voice channel with hands-free push-to-talk for talking through a session without breaking it. We also describe the session tools and in-headset tutorials that keep a room full of personalized instruments and new performers from tipping into chaos. Together, these features move CsoundMeta from a system performers play inside toward a system they compose, build, and record in, one shaped as much by the Csound community as by its designers.

Keywords: Unity, CsoundUnity, Meta Quest, VR, AR, XR, Cabbage, Blockade Labs, World Generation, Low-Latency Audio, Sampler, Personalization, Multiplayer

1 Introduction

CsoundMeta was introduced as an immersive, multiplayer environment for Csound performance, built in Unity for Meta Quest headsets, where sounds live as physical objects that players can strike, grab, stretch, and toss [1]. That first system already let a room full of performers reach into a shared virtual or physical space and touch a fixed set of instruments and worlds, and a performer could already create or bring in samples, though each stayed local to that headset and importing an outside one was tedious. What it did not yet do was let a Csound instrument itself travel from a desk into the room, or let anything a performer brought in reach anyone else’s headset: an instrument built at a desk stayed on the desk, and the worlds performers played inside were chosen from a curated list, not shaped by the people standing in them.

Section 2 covers the import pipeline that lets any Csounder bring a `.csd` instrument or Cabbage preset into the headset, or now a browser, as a new Orb, Wanderer, LiveFX, or Classic. Section 3 covers World generation, now a full in-headset authoring loop shared by everyone in a room or across

a continent. Section 4 covers the most consequential change: a rebuilt low-latency audio engine, a Mic instrument now also usable as a shared microphone, a spatialized voice channel, and a sampler that can record from anything in the patch with real-time overdub. Section 5 covers the tools, modes, and tutorial library that keep a personalized session from collapsing into chaos. Our hope, as it was two years ago, is that CsoundMeta becomes less a piece of software we hand to the Csound community and more a shared instrument the community builds together.

2 Bringing Your Own Instruments Into the Room

The heart of this update is an import pipeline that no longer cares where a Csound instrument was built: from the headset’s own file browser, a performer can pick up any `.csd` file on their Quest and have it appear as a new Orb or Wanderer sound, no recompiling and no desktop tether required, and a companion path imports Cabbage `.snaps` files directly as ready-to-use presets, with a review screen for renaming each before it lands in the palette.

As of our most recent release, the import pipeline no longer requires the headset at all: a companion web application lets a performer upload a `.csd`, or a full folder or `.zip` containing its samples, includes, and tables, from any browser, with Cabbage presets detected and offered for renaming before publishing, and choose whether it should behave as an Orb, a Classic, or an FX so it appears only in the picker that can actually play it. Because compilation failures that are silent in-headset are often the hardest part of bringing outside work into CsoundMeta, every upload is checked by a purpose-built version of our Dr.C agent [11,12,13,14], which reads the instrument’s own compile output and helps design, modify, and debug a `.csd` that will not yet run, so a performer can see exactly why an instrument that worked on their desktop won’t yet play on someone else’s head.

Reordering, renaming, deleting, and now searching, a management layer once specific to Classics (our name for longer, more elaborate patches) now covers every object type a performer can own, Orbs, Wanderers, LiveFX, Samples, Palettes, Movements, Worlds, and Compositions, each with per-user filtering so a crowded session does not bury one performer’s sounds under everyone else’s. Every imported instrument, and every Orb, Wanderer, Sampler, and effect built from it, keeps a distinct shape and color so a room full of personalized objects still reads at a glance (Fig. 1a), and every deletion now runs a dependency check first, protecting a preset, palette, or saved Movement that depends on it.

Imported sounds and presets are not private to the performer who brought them in. Once in a session they sync live to every connected client and persist across sessions, so one Csounder’s original instrument, or a factory sample pulled into a sampler (Fig. 1b), becomes something the whole ensemble can reach for, not only its author.

This shift relocates authorship of the shared sonic environment from the system’s designers to its performers. In the original CsoundMeta, every performer in a room drew from the same fixed instrument set; a session’s musical identity was decided in advance, by us. Once any performer’s own `.csd` or Cabbage preset can enter the shared library and persist across sessions, the instrument palette becomes a running record of who has played in that room, not a specification we wrote. This changes what “the same instrument” means in an ensemble: two performers striking an Orb may be triggering one performer’s own patch, built at a desk days earlier, rather than a factory sound both already know. We treat this as the more interesting design problem raised by the update, more than the mechanics of the import pipeline itself.

3 Prompting Your Own Worlds, Together

Where the original system offered performers a curated set of Blockade Labs worlds to choose from [1,10], the current system lets any performer generate that world themselves, from inside the headset, in the moment (Fig. 2). A short wizard walks through prompt, negative prompt, style, and an optional seed, with a floating panel showing generation progress and an estimated time. Generated Worlds can be remixed, renamed, or deleted, using the same dependency check from Section 2 to warn if one is still placed on a saved timeline, and can be repositioned and rotated in place, so a stage arrangement built and rehearsed at home lines up exactly once the same World is loaded on the performance stage.

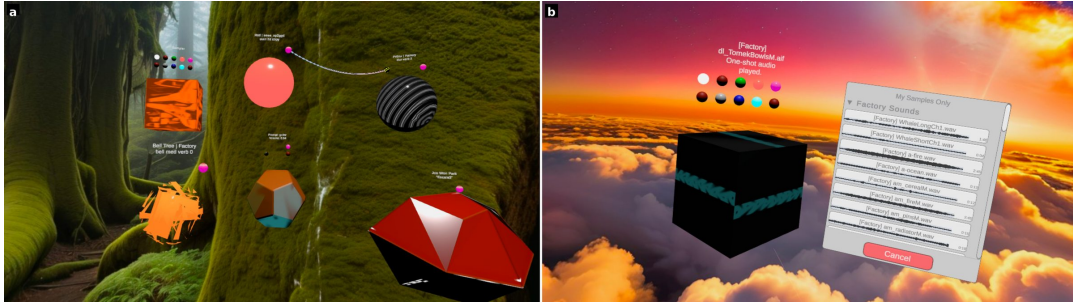


Fig. 1. (a) Imported and generated instruments, each with a distinct shape and color, connected with patch cords across a shared world, including an instrument brought in by another performer. (b) A Sampler browsing factory audio to load as a new one-shot sound.

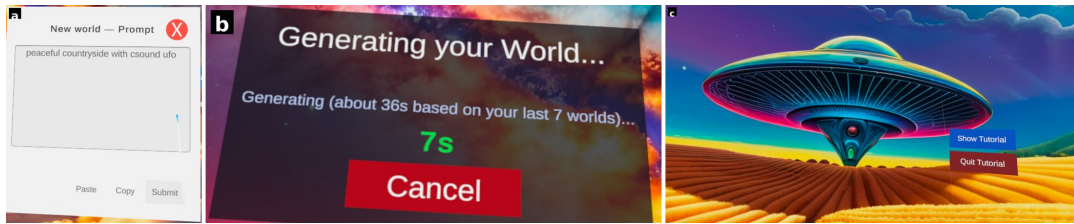


Fig. 2. Generating a world from inside the headset: (a) enter a prompt, (b) watch it render with a live time estimate, (c) arrive in the finished World.

Every generated World syncs automatically to all connected clients the moment it finishes rendering, so a World one performer prompts into being becomes the room everyone is standing in, colocated or remote. Once there, performers can hand instruments to one another mid-jam, an Orb, a Wanderer, or the new Mic from Section 4, tossed from one performer’s hands into another’s across the room or across an ocean, and it keeps sounding, synced, the whole way there. Built-in Worlds now live in the same cloud-backed storage as user-generated ones, trimming the application’s memory footprint.

We treat the World as a shared compositional parameter, the same way we already treat an Orb’s shape as one, because both are objects a performer can act on mid-performance and have the result heard or seen by everyone else immediately. A world chosen in advance is a setting; a world prompted, remixed, and renamed inside the performance is closer to a slow-moving visual voice in the ensemble, one whose “notes” are prompts instead of pitches. Making World generation synchronous and in-headset was a precondition for this: a world that has to be prepared beforehand by someone outside the performance cannot function as an instrument played during it. The result is a World system that behaves less like a settings menu and more like an instrument in its own right: performers do not just choose the space they play in, they compose it, and they compose it as an ensemble.

4 Capturing and Looping Sound in Real Time

The most recent and, for live ensemble work, the most important change is a rebuilt audio input pipeline. Live effects now run on a native audio engine that carries the signal from microphone to effect to speaker outside Unity’s own audio thread, fast enough that a performer singing or speaking into the room no longer feels any lag between mouth and ear. We have not measured this formally: we judge it the way musicians judge a signal chain, by ear, in real rehearsal and performance, where it now also handles real-time capture, playback, and processing. Two separate implementations of this engine, one for Quest and one for Windows and the Unity Editor, produce the same result by different means, so effects built and tested at a desk behave the same way once strapped to a headset, and Chained LiveFX Orbs now run end to end on either one.

Where earlier versions grabbed the hardware microphone silently, the microphone is now its own object: a dedicated Mic instrument, spawned from the instrument menu, that a performer brings close to their mouth to activate, the same gesture as picking up a handheld mic, and switches on and

off with a button. The Mic’s body doubles as a VU meter, and, like any Orb, can be handed from performer to performer mid-session rather than staying tethered to whoever spawned it. A single Mic can feed several LiveFX Orbs at once, only ever the ones a performer has explicitly patched it into, and to keep one runaway effect from dragging down a whole session, the engine watches CPU and memory across its live-effect slots and evicts a single misbehaving effect rather than the room.

Alongside the performance Mic described above, a separate spatialized voice channel lets performers talk to each other during a session without pulling a headset off, activated hands-free by raising a hand to the mouth so it never competes with two-handed play. A single Mic instrument can also serve as one shared microphone for every client’s effects at once: whoever is nearest becomes its capturer, and the handoff to a new capturer is automatic as performers change position.

The sampler received a matching overhaul: recording is now input-driven, low-latency, and overdub-capable, so a performer can loop a phrase and layer new takes in real time, like a looping pedal on a stage. Its input is no longer fixed to the built-in microphone: it can record from the new Mic or any other instrument in the connection graph, another performer’s Orb, a Classic, even another sampler, and every recording uploads to the cloud the moment it is saved.

5 Keeping a Personalized Session Together

Giving performers more to bring into a session raises an obvious risk: a room full of personalized instruments, worlds, and live sampling can turn loud and chaotic fast. A growing set of tools manages that risk without taking the personalization away.

Two host-facing modes keep a session legible as it grows. Jam Mode caps how many objects a client can create, so a beginner cannot spawn or delete objects fast enough to cause chaos, with an exemption for the host. Concert Mode hides each instrument’s record, load, save, and delete controls behind a host-controlled volume lock and label toggles, for a recital or rehearsal where the audience should not see who is playing (Fig. 3a). A companion Chat Mode, built on the voice channel from Section 4, lets performers talk through a plan mid-rehearsal without pulling their headsets off.

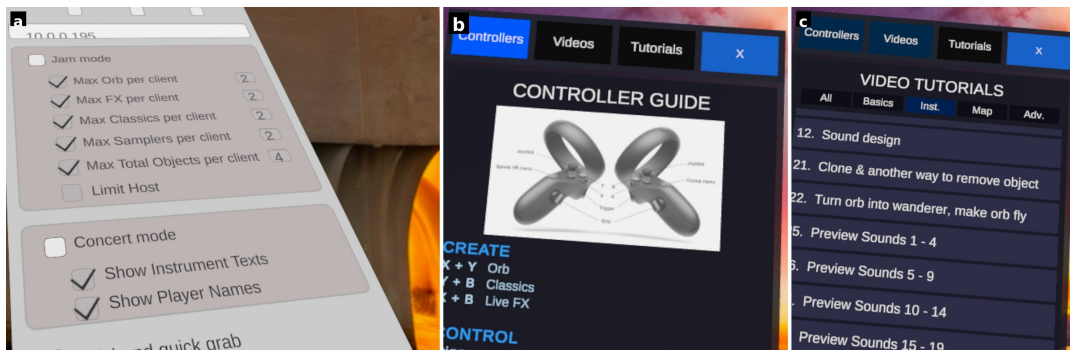


Fig. 3. (a) Jam Mode and Concert Mode host settings, capping objects per client and controlling label and instrument-control visibility. (b) The in-headset controller guide. (c) The categorized video tutorial library.

New performers are not left to guess: an in-headset panel covers the controller layout (Fig. 3b) and links to a categorized video tutorial library (Fig. 3c), so a Csounder can learn the system without leaving VR.

6 Conclusion and Next Steps

Two years ago we ended our first CsoundMeta paper hoping performers could import their own .csd files, generate worlds shaped by what they play, and share sounds across a room or the world; each hope now has a working implementation, from browser-based imports debugged by Dr.C, to instantly shared worlds, to a cloud-synced sampler. What began as a curated system is becoming one the community builds together: next steps include a World that reacts to live audio and a WebVR/WebXR version of CsoundMeta, so joining a session, at least as a listener, needs no headset at all.

References

1. S. Bear and R. Boulanger. Csound in the MetaVerse – From Cabbage to CsoundUnity and Beyond: Developing a Working Environment for SoundScapes, SoundCollages, and Collaborative SoundPlay. In *Proceedings of the 7th International Csound Conference (ICSC 2024)*, S. Stojak and A. Hofmann (eds.), University of Music and Performing Arts Vienna (mdw), pp. 115–120, 2024.
2. C. Çakmak, A. Çamcı, and A. G. Forbes. Networked Virtual Environments as Collaborative Music Spaces. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*, Brisbane, Australia, 2016.
3. S. Serafin, C. Erkut, J. Kojs, N. C. Nilsson, and R. Nordahl. Virtual Reality Musical Instruments: State of the Art, Design Principles, and Future Directions. *Computer Music Journal*, 40(3):22–40, 2016.
4. R. Walsh. Cabbage Audio Documentation. <https://cabbageaudio.com/docs/introduction/>, 2024.
5. R. Walsh. CsoundUnity. <https://github.com/rorywalsh/CsoundUnity>, 2024.
6. Unity Technologies. Unity Real-Time 3D Development Platform. <https://unity.com/>, 2024.
7. Meta. Meta Quest. <https://www.meta.com/quest/>, 2024.
8. ZeroTier, Inc. ZeroTier Networking. <https://www.zerotier.com/>, 2024.
9. Mirror Networking. Mirror Networking for Unity. <https://mirror-networking.com/>, 2024.
10. Blockade Labs. Skybox AI. <https://skybox.blockadelabs.com/>, 2024.
11. M. Larrea and R. Boulanger. Introducing Dr.C. In *Proceedings of the 8th International Csound Conference (ICSC)*, Trapani, Italy, 2026. To appear.
12. R. Boulanger and M. Larrea. Dr.C: A Human-in-the-Loop Agentic AI for Csound. In *Proceedings of the Linux Audio Conference (LAC)*, Maynooth University, Ireland, 2026.
13. M. Larrea and R. Boulanger. Dr.C Standalone. <https://github.com/mateolarreaferro/DRC-Standalone>, 2026.
14. M. Larrea and R. Boulanger. Dr.C (Command-Line Interface). <https://github.com/mateolarreaferro/Dr.C>, 2026.